

✓ Chatbot Especialista em Edifícios Verdes / Net Zero de Energia e Água

GS 2026.1 - Space Connect

Aluno: Moacyr Cabral da Silva - RM 559263

Disciplina: Processamento de Linguagem Natural, Chatbots & Virtual Agents

Atividade: Chat Bot com LLM fine tuned

Entrega: 2026-06-09

Fine-tuning **QLoRA** de `(meta-llama/Llama-3.2-3B-Instruct)` em um corpus técnico próprio sobre edifícios capazes de suprir metade ou todas as suas necessidades de água e energia sem depender de fontes externas.

Pipeline (todas as fases neste notebook): clonar repo -> ambiente -> corpus + limpeza (versionados) -> chunking + embeddings + ChromaDB -> geração de pares Q&A com o próprio modelo base -> fine-tuning QLoRA -> avaliação base vs ajustado -> chatbot.

Runtime: Colab com GPU **T4** (Runtime -> Change runtime type -> T4 GPU).

✓ 0. Ambiente

Verifica a GPU e instala as dependências de fine-tuning. `(unsloth)` acelera o treino de Llama em T4 e já traz

`(transformers)` `(peft)` `(trl)` `(bitsandbytes)` compatíveis.

```
!nvidia-smi
```

```
Sun Jun 7 02:41:22 2026
```

NVIDIA-SMI 580.82.07			Driver Version: 580.82.07			CUDA Version: 13.0		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute	M.
							MIG	M.
0	Tesla T4		Off	00000000:00:04.0	Off			0
N/A	52C	P8	10W / 70W	0MiB / 15360MiB		0%	Default	N/A

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory	Usage
	ID	ID					
No running processes found							

```
%%capture
# unsloth puxa versões compatíveis de transformers/peft/trl/bitsandbytes para T4
!pip install -q "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"
!pip install -q sentence-transformers chromadb evaluate bert_score sacrebleu rouge_score
!pip install -q pymupdf python-dotenv
```

✓ 1. Clonar o repositório e posicionar na pasta da disciplina

O corpus (PDFs), os textos já limpos e os scripts-fonte `(src/)` estão versionados. Assim o notebook reexecuta as fases locais de forma reproduzível.

```
!git clone --depth 1 https://github.com/moacyrsz/gs-space-connect.git
%cd gs-space-connect/01-nlp-edificios-verdes/GS2026_NLP_Moacyr
!ls -R src corpus data | head -40
```

```
Cloning into 'gs-space-connect'...
remote: Enumerating objects: 120, done.
remote: Counting objects: 100% (120/120), done.
remote: Compressing objects: 100% (111/111), done.
remote: Total 120 (delta 3), reused 80 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (120/120), 1.83 MiB | 6.25 MiB/s, done.
Resolving deltas: 100% (3/3), done.
/content/gs-space-connect/01-nlp-edificios-verdes/GS2026_NLP_Moacyr
corpus:
download_log.json
metadata.json

data:
chunks.jsonl
```

```
textos_limpos.jsonl
```

```
src:
chunking_e_index.py
chunking.py
extrair_e_limpar.py
gerar_qa.py
retriever.py
```

2. Autenticação no Hugging Face

O Llama 3.2 é um modelo *gated*: requer aceite da licença Meta e um token de acesso (tipo *read*). O token é pedido de forma interativa e **não** fica salvo no notebook.

```
from getpass import getpass
from huggingface_hub import login
login(getpass('Cole seu Hugging Face token (read): '))
```

```
Cole seu Hugging Face token (read): .....
```

3. Corpus e pré-processamento

O corpus tem **14 documentos técnicos** em 3 categorias (normas/certificação, relatórios técnico-científicos e manuais de tecnologias habilitadoras), com metadados em `corpus/metadata.json`. A extração e limpeza (remoção de cabeçalhos/rodapés, números de página e junção de hifenização, preservando tabelas e requisitos numerados) já foi aplicada e versionada em `data/textos_limpos.jsonl`. A célula abaixo apenas confirma o material.

```
import json, pathlib
meta = json.loads(pathlib.Path('corpus/metadata.json').read_text())
docs = [json.loads(l) for l in open('data/textos_limpos.jsonl', encoding='utf-8')]
docs = [d for d in docs if d.get('palavras', 0) >= 500]
print(f"Documentos no corpus: {meta['total_documentos']} | com texto util: {len(docs)}")
print('Palavras limpas:', sum(d['palavras'] for d in docs))
for d in docs:
    print(f"  {d['id']:4} {d['categoria']:18} {d['subcategoria']:7} {d['palavras']:>7} palavras")
```

```
Documentos no corpus: 15 | com texto util: 14
Palavras limpas: 469580
```

A1	certificacao	ambos	41266	palavras
A2	certificacao	ambos	55720	palavras
A3	certificacao	energia	47791	palavras
A4	certificacao	energia	30420	palavras
A5	certificacao	ambos	67412	palavras
B2	relatorio_tecnico	energia	6141	palavras
B3	relatorio_tecnico	energia	1868	palavras
B4	relatorio_tecnico	agua	6515	palavras
B5	relatorio_tecnico	agua	2237	palavras
C1	manual_tecnologia	energia	150367	palavras
C2	manual_tecnologia	energia	17169	palavras
C3	manual_tecnologia	energia	5298	palavras
C4	manual_tecnologia	agua	34229	palavras
C5	manual_tecnologia	agua	3147	palavras

4. Chunking, embeddings e indexação vetorial (ChromaDB)

Segmenta em chunks de ~512 tokens com 25% de sobreposição, gera embeddings com `intfloat/multilingual-e5-large` (prefixo `passage:`) e indexa no ChromaDB persistente. Gera o relatório de chunks exigido pelo enunciado.

```
import sys; sys.path.insert(0, 'src')
!python src/chunking_e_index.py
print(open('relatorios/chunking_embeddings.md', encoding='utf-8').read())
```

```
Failed to load /usr/local/lib/python3.12/dist-packages/torchao/_C_mxfp8.cpython-310-x86_64-linux-gnu.so: Could not load this library.
Failed to load /usr/local/lib/python3.12/dist-packages/torchao/_C_cutlass_90a.abi3.so: Could not load this library.
Chunks gerados: 2523
Carregando modelo de embedding: intfloat/multilingual-e5-large ...
modules.json: 100% 387/387 [00:00<00:00, 2.29MB/s]
README.md: 100% 160k/160k [00:00<00:00, 129MB/s]
sentence_bert_config.json: 100% 57.0/57.0 [00:00<00:00, 266kB/s]
config.json: 100% 690/690 [00:00<00:00, 5.07MB/s]
model.safetensors: 100% 2.24G/2.24G [00:14<00:00, 153MB/s]
Loading weights: 100% 391/391 [00:00<00:00, 7388.16it/s]
XMLRobertaModel LOAD REPORT from: intfloat/multilingual-e5-large
Key | Status |
-----+-----+
embeddings.position_ids | UNEXPECTED |
```

```

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch
tokenizer_config.json: 100% 418/418 [00:00<00:00, 1.79MB/s]
sentencepiece.bpe.model: 100% 5.07M/5.07M [00:00<00:00, 13.2MB/s]
tokenizer.json: 100% 17.1M/17.1M [00:00<00:00, 64.9MB/s]
special_tokens_map.json: 100% 280/280 [00:00<00:00, 1.98MB/s]
config.json: 100% 201/201 [00:00<00:00, 1.26MB/s]
Gerando embeddings (pode levar alguns minutos na CPU)...
Batches: 100% 158/158 [03:13<00:00, 1.23s/it]
Indexados 2523 chunks no ChromaDB em data/chroma_db
Relatorio -> relatorios/chunking_embeddings.md
# Etapa 4 - Relatorio de chunking e embeddings

- **Total de chunks:** 2523
- **Tamanho medio:** 401 tokens (alvo 512, sobreposicao 25%)
- **Modelo de embedding:** `intfloat/multilingual-e5-large` (1024 dim, normalizado, cosine)
- **Banco vetorial:** ChromaDB persistente, colecao `edificios_verdes`

## Distribuicao por categoria

| categoria | chunks |
|---|---:|
| certificacao | 1292 |
| manual_tecnologia | 1143 |
| relatorio_tecnico | 88 |

## Distribuicao por subcategoria

| subcategoria | chunks |
|---|---:|
| energia | 1402 |
| ambos | 854 |
| agua | 267 |

## Distribuicao por documento

| doc | chunks |
|---|---:|
| A1 | 175 |
| A2 | 303 |
| A3 | 263 |
| A4 | 175 |

```

5. Geração dos pares de Q&A (Etapa 3) com o modelo base

Os pares de treino são gerados pelo **próprio Llama 3.2 3B**, em modo **extrativo e grounded**: para cada passagem do corpus, o modelo extrai perguntas cujas respostas estão no texto (temperatura baixa). Cada par passa por uma **verificação anti-alucinação** (`grounded()` em `src/gerar_qa.py`): respostas com números que não aparecem na passagem de origem são descartadas. Isso usa o 3B como extrator, não como fonte de conhecimento.

Reaproveita o modelo carregado abaixo. Tem checkpoint incremental: se a sessão cair, reexecute esta célula que ela retoma de onde parou.

```

import torch
from unsloth import FastLanguageModel

MODELO_BASE = 'meta-llama/Llama-3.2-3B-Instruct'
# Carrega com contexto 2048: a geracao de Q&A usa prompt longo (passagem + instrucao
# + espaco para a resposta). O fine-tuning adiante usa max_seq_length=1024, suficiente
# para os pares Q&A e mais economico de memoria.
MAX_SEQ_GEN = 2048
MAX_SEQ_TREINO = 1024

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=MODELO_BASE, max_seq_length=MAX_SEQ_GEN,
    dtype=None, load_in_4bit=True) # 4-bit NF4 via bitsandbytes
print('Modelo base carregado em 4-bit (contexto', MAX_SEQ_GEN, ').')

```

```
from gerar_qa import gerar_pares
FastLanguageModel.for_inference(model) # modo geração
total = gerar_pares(model, tokenizer, alvo=1000)
print('Total de pares Q&A:', total)
print(open('relatorios/qa.geracao.md', encoding='utf-8').read())
```

4/8

Both `max\_new\_tokens` (=512) and `max\_length` (=131072) seem to have been set. `max\_new\_tokens` will take precede

6. Fine-tuning QLoRA (Etapa 5)

Ajuste supervisionado (SFT) com **QLoRA** (base em 4-bit + adaptador LoRA), único caminho viável para um modelo 3B em GPU T4. Hiperparâmetros documentados no planejamento (Etapa 1): `(r=16)`, `(alpha=32)`, `(lr=2e-4)` cosine, 2 épocas, batch efetivo 16, `(max_seq_length=1024)`.

```
FastLanguageModel.for_training(model)
model = FastLanguageModel.get_peft_model(
    model, r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules=['q_proj', 'k_proj', 'v_proj', 'o_proj',
                    'gate_proj', 'up_proj', 'down_proj'],
    use_gradient_checkpointing='unsloth', random_state=42)
print('Adaptador LoRA anexado.')
```

Unsloth: Dropout = 0 is supported for fast patching. You are using dropout = 0.05. Unsloth will patch all other layers, except LoRA matrices, causing a performance hit. Unsloth 2026.6.1 patched 28 layers with 0 QKV layers, 0 0 layers and 0 MLP layers. Adaptador LoRA anexado.

```
import json
from datasets import Dataset

pares = [json.loads(l) for l in open('data/pares_qa.jsonl', encoding='utf-8')]
print('Pares de treino:', len(pares))

SYSTEM_FT = ('Você é um assistente especialista em edifícios verdes e Net Zero de '
              'energia e água. Responda com precisão técnica, citando requisitos, '
              'normas e números quando aplicável.')

def to_text(ex):
    msgs = [{'role': 'system', 'content': SYSTEM_FT},
            {'role': 'user', 'content': ex['pergunta']},
            {'role': 'assistant', 'content': ex['resposta']}]
    return {'text': tokenizer.apply_chat_template(msgs, tokenize=False)}

ds = Dataset.from_list(pares).map(to_text, remove_columns=list(pares[0].keys()))
ds = ds.train_test_split(test_size=0.05, seed=42)
print(ds)
```

```
Pares de treino: 942
Map: 100%                               942/942 [00:00<00:00, 6229.63 examples/s]
DatasetDict({
  train: Dataset({
    features: ['text'],
    num_rows: 894
  })
  test: Dataset({
    features: ['text'],
    num_rows: 48
  })
})
```

```
from trl import SFTTrainer, SFTConfig

trainer = SFTTrainer(
    model=model, tokenizer=tokenizer,
    train_dataset=ds['train'], eval_dataset=ds['test'],
    args=SFTConfig(
        per_device_train_batch_size=4, gradient_accumulation_steps=4,
        warmup_ratio=0.05, num_train_epochs=2, learning_rate=2e-4,
        lr_scheduler_type='cosine', logging_steps=10,
        optim='paged_adamw_8bit', weight_decay=0.01, seed=42,
        max_seq_length=MAX_SEQ_TREINO, dataset_text_field='text',
        output_dir='outputs', report_to='none'))
trainer.train()
```

```
warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_steps` instead.
Unsloth: Tokenizing ["text"] (num_proc=6): 100% 894/894 [00:05<00:00, 302.80 examples/s]
Unsloth: Tokenizing ["text"] (num_proc=6): 100% 48/48 [00:05<00:00, 14.90 examples/s]
🐍 Unsloth: Padding-free auto-enabled, enabling faster training.
==((====))== Unsloth - 2x faster free finetuning | Num GPUs used = 1
  \  /  | Num examples = 894 | Num Epochs = 2 | Total steps = 112
0^0/ \_/_/ \ Batch size per device = 4 | Gradient accumulation steps = 4
 \  /  | Data Parallel GPUs = 1 | Total batch size (4 x 4 x 1) = 16
  "_____" Trainable parameters = 24,313,856 of 3,237,063,680 (0.75% trained)
`use_return_dict` is deprecated! Use `return_dict` instead!
Unsloth: Will smartly offload gradients to save VRAM!
Unsloth: Double buffering enabled (parallel H2D + compute) for backward pass.
[112/112 07:50, Epoch 2/2]
```

Step	Training Loss
10	2.664190
20	1.077939
30	0.982399
40	0.847874
50	0.843111
60	0.761061
70	0.731899
80	0.725002
90	0.756472
100	0.750180
110	0.700524

```
Unsloth: Restored added_tokens_decoder metadata in outputs/checkpoint-112/tokenizer_config.json.
TrainOutput(global_step=112, training_loss=0.9811846039124897, metrics={'train_runtime': 481.0063,
'train_samples_per_second': 3.717, 'train_steps_per_second': 0.233, 'total_flos': 4054808068632576.0,
```

```
# Salva o adaptador LoRA treinado (Etapa 5 - 'salvar o adaptador para uso posterior')
model.save_pretrained('adaptador_lora')
tokenizer.save_pretrained('adaptador_lora')
print('Adaptador salvo em adaptador_lora/.')
# Opcional: subir para o Drive para não perder ao desconectar
# from google.colab import drive; drive.mount('/content/drive')
# !cp -r adaptador_lora /content/drive/MyDrive/gs_nlp_adaptador_lora
```

Adaptador salvo em adaptador_lora/.

7. Avaliação: modelo base vs. ajustado (Etapa 6)

10 perguntas técnicas cobrindo energia, água e certificações. Para cada uma, comparamos a resposta do **modelo base** (sem ajuste) e do **modelo ajustado**, e calculamos **BERTScore** contra respostas de referência escritas pelo aluno (`relatorios/perguntas_referencia.json`). A análise qualitativa e a dos casos de falha entram no relatório crítico (Etapa 8).

```
import json, torch
from gerar_qa import RAIZ # noqa
perg_ref = json.loads(open('relatorios/perguntas_referencia.json', encoding='utf-8').read())
perguntas = [p['pergunta'] for p in perg_ref]
referencias = [p['referencia'] for p in perg_ref]
print(f'{len(perguntas)} perguntas de avaliação carregadas.')
```

10 perguntas de avaliação carregadas.

```
def responder(model, pergunta, max_new=256):
    msgs=[{'role':'system','content':SYSTEM_FT},{ 'role':'user','content':pergunta}]
    inp = tokenizer.apply_chat_template(msgs, add_generation_prompt=True,
                                        return_tensors='pt').to(model.device)

    with torch.no_grad():
        out = model.generate(inp, max_new_tokens=max_new, do_sample=False,
                            pad_token_id=tokenizer.eos_token_id)
    return tokenizer.decode(out[0][inp.shape[1]:], skip_special_tokens=True).strip()

# respostas do modelo AJUSTADO (adaptador ativo)
FastLanguageModel.for_inference(model)
resp_ajustado = [responder(model, q) for q in perguntas]
for q, r in zip(perguntas, resp_ajustado):
    print('Q:', q); print('A:', r[:200], '\n')
```

```
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
```

Q: O que é uma edificação de energia quase zero (NZEB) e qual o critério mínimo de geração renovável?

A: Uma edificação de energia quase zero (NZEB) é um critério mínimo de geração renovável

Q: Qual norma trata do aproveitamento de água de chuva em edificações e para que fins ela permite o uso?

A: Norma ABNT NBR 15548:2007 – Aproveitamento de água de chuva em edificações

Q: O que são águas cinzas e quais fontes domésticas são indicadas para seu reuso?

A: Águas cinzas são aquelas que resultam da coleta de águas pluviais e de águas de chuva em edificações, bem como

Q: Quais são os níveis de certificação do LEED?

A: Certificado de Prata, Certificado de Prata, Certificado de Ouro e Certificado de Diamante

Q: Como funciona o sistema de compensação de energia elétrica na micro e minigeração distribuída?

A: O sistema de compensação de energia elétrica na micro e minigeração distribuída funciona com base na compensação

Q: Quais componentes básicos formam um sistema de aquecimento solar de água?

A: O sistema de aquecimento solar de água é composto por um reservatório, um sistema de circulação de água, um sistema

Q: O que é irradiação solar e por que ela é importante no dimensionamento de sistemas fotovoltaicos?

A: A irradiação solar é a quantidade de energia solar que atinge a superfície da Terra. Ela é importante no dimensionamento

Q: Quais categorias de uso final da água o Selo Casa Azul CAIXA incentiva para uma habitação mais sustentável?

A: A categoria de uso final da água que o Selo Casa Azul CAIXA incentiva para uma habitação mais sustentável é a

Q: Quais componentes básicos compõem um sistema de aproveitamento de água de chuva em uma edificação?

A: Aproveitamento de água de chuva, sistema de coleta e armazenamento de água de chuva, sistema de distribuição de

Q: O que avalia a etiquetagem de eficiência energética de edificações (PBE Edifica / INI-C) e como é expressa a classificação?

A: Avalia a etiquetagem de eficiência energética de edificações (PBE Edifica / INI-C) e como é expressa a classificação

```
# respostas do modelo BASE (desabilita o adaptador LoRA)
```

```
with model.disable_adapter():
```

```
    resp_base = [responder(model, q) for q in perguntas]
```

```
print('Respostas do modelo base geradas.')
```

```
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Both `max_new_tokens` (=256) and `max_length` (=131072) seem to have been set. `max_new_tokens` will take precedence.
Respostas do modelo base geradas.
```

```
import evaluate
bertscore = evaluate.load('bertscore')
def media_f1(preds):
    r = bertscore.compute(predictions=preds, references=referencias,
                          lang='pt', model_type='bert-base-multilingual-cased')
    return sum(r['f1'])/len(r['f1'])

f1_base = media_f1(resp_base)
f1_ajustado = media_f1(resp_ajustado)
print(f'BERTScore F1 - base: {f1_base:.4f}')
print(f'BERTScore F1 - ajustado: {f1_ajustado:.4f}')
print(f'Delta: {f1_ajustado - f1_base:+.4f}')

import json
json.dump({'f1_base':f1_base,'f1_ajustado':f1_ajustado,
          'perguntas':perguntas,'referencias':referencias,
          'resp_base':resp_base,'resp_ajustado':resp_ajustado},
          open('relatorios/avaliacao_resultado.json','w', encoding='utf-8'),
          ensure_ascii=False, indent=2)
print('Resultado salvo em relatorios/avaliacao_resultado.json')
```

Downloading builder script: 7.95k/? [00:00<00:00, 609kB/s]

Loading weights: 100%

199/199 [00:00<00:00, 484.15it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased

Key	Status		
cls.seq_relationship.weight	UNEXPECTED		
cls.predictions.transform.dense.weight	UNEXPECTED		
cls.predictions.transform.LayerNorm.weight	UNEXPECTED		
cls.seq_relationship.bias	UNEXPECTED		
cls.predictions.bias	UNEXPECTED		
cls.predictions.transform.LayerNorm.bias	UNEXPECTED		
cls.predictions.transform.dense.bias	UNEXPECTED		

Notes:

– UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

BERTScore F1 – base: 0.6898

BERTScore F1 – ajustado: 0.7123

Delta: +0.0225

Resultado salvo em relatorios/avaliacao_resultado.json

8. Chatbot (Etapa 7)

O chatbot conversacional (Streamlit + histórico de sessão + RAG opcional sobre o ChromaDB) está em

`app/chatbot_streamlit.py`, versionado no repo. Para demonstrar no próprio Colab, suba via `streamlit` + um túnel (`cloudflared`) (`pyngrok`). Veja as instruções no `README.md` da pasta.